

Below is a **MASTER TABLE** of software development methodologies used in the IT industry, with **when to use which methodology** (best-fit situations).

✓ Master Table — Software Methodologies in IT Industry (When to Use Which)

#	Methodology	Type / Category	Best used when... (Ideal Situations)	Not suitable when...	Real industry examples
1	Waterfall	Traditional / Linear	Requirements are fixed , scope is stable, high documentation needed	Requirements change frequently	Government projects, construction-like systems
2	V-Model (Verification & Validation)	Traditional / Quality-focused	Testing must be planned from day 1, compliance-heavy projects	Frequent requirement changes	Healthcare software, aerospace
3	Incremental Model	Traditional + Iterative	You want to deliver in small releases quickly	Architecture unclear	Banking modules, internal enterprise apps
4	Iterative Model	Iterative	Product can evolve through feedback	Fixed requirement expectation	Product startups, UI-based applications
5	Spiral Model	Risk-driven	Risk is high, complex system, you need risk analysis each cycle	Low budget, small projects	Defense systems, research products
6	Prototype Model	Exploratory / UX-heavy	Requirements unclear, need user validation early	Highly regulated fixed scope	Customer apps, new UI/UX, portals
7	RAD (Rapid Application Development)	Fast delivery	Need quick working product with intense user involvement	Large complex projects with heavy backend	CRM apps, internal tools
8	Agile (Generic)	Agile umbrella	Requirements change frequently, need frequent releases	Fixed scope/time with heavy compliance	Most modern IT companies

9	Scrum	Agile framework	Team wants structured Agile with sprints & roles	Too many dependencies or non-committed teams	SaaS products, fintech apps
10	Kanban	Lean + Agile	Continuous flow work, support/maintenance	Fixed sprint targets required	DevOps, support teams, bug fixing
11	Scrumban	Hybrid Agile	Mix of Scrum structure + Kanban flow	Strict Scrum needed	Fast-moving product teams
12	Extreme Programming (XP)	Agile engineering	High-quality code needed, fast changes, heavy testing	Team not disciplined in engineering	APIs, fintech core services
13	Lean Software Development	Lean	Reduce waste, optimize process efficiency	Teams need strict structure	Manufacturing IT, operational systems
14	Crystal	Agile lightweight	Small teams, low criticality apps	Very large projects	Small web apps
15	DSDM (Dynamic System Development Method)	Agile + governance	Time and budget fixed, scope flexible	Teams dislike governance	Enterprise delivery
16	FDD (Feature Driven Development)	Agile structured	Large systems needing feature-based planning	Rapid unpredictable requirements	Enterprise ERP modules
17	SAFe (Scaled Agile Framework)	Scaled Agile	Many Agile teams, enterprise scale, portfolio management	Small teams	Banking, telecom, large IT org
18	LeSS (Large Scale Scrum)	Scaled Scrum	Many teams building one product using Scrum	Highly distributed teams	Large single-product orgs

19	Nexus (Scaled Scrum)	Scaled Scrum	Multiple Scrum teams with shared backlog	Highly independent teams	Large Scrum programs
20	Spotify Model	Agile org model	Autonomous squads/tribes, innovation culture	Strict compliance needed	Product tech companies
21	DevOps	Delivery methodology	Fast deployments + automation + collaboration	Org separation is rigid	Cloud products, CI/CD orgs
22	DevSecOps	DevOps + Security	Security must be integrated into CI/CD	Security is afterthought culture	Banking, fintech, cloud security
23	CI/CD-based Delivery	DevOps practice	Frequent automated testing + releases	Legacy systems no automation	Modern web/mobile apps
24	RUP (Rational Unified Process)	Iterative + heavy documentation	Enterprise & complex systems, need governance	Small fast projects	Enterprise apps, large-scale systems
25	Unified Process (UP)	Iterative	Architecture-focused iterative build	Very small projects	Enterprise architecture projects
26	Cleanroom Software Engineering	Quality / defect prevention	Zero-defect critical software	Agile speed required	Defense, high safety systems
27	Big Bang Model	Informal	Small experiments, learning projects	Anything professional/critical	Student projects, prototype demos
28	Component-Based Development (CBD)	Reuse-driven	Build system from reusable components	Unique custom system from scratch	Enterprise systems, UI frameworks
29	SOA (Service Oriented Architecture approach)	Architecture methodology	Enterprise integration, reusable services	Small standalone apps	Large banks, legacy modernization

30	Microservices Delivery Model	Architecture + methodology	High scalability, independent deployment needed	Small teams, low maturity	Netflix-style systems, ecommerce
31	DDD (Domain Driven Design)	Design methodology	Complex domain logic, business-critical systems	Simple CRUD apps	Banking, insurance, ERP
32	TDD (Test Driven Development)	Engineering methodology	High reliability, API-driven dev	Teams don't follow test discipline	APIs, libraries, fintech
33	BDD (Behavior Driven Development)	Engineering + collaboration	Strong business + QA collaboration required	Poor business involvement	Product development orgs
34	ATDD (Acceptance Test Driven Development)	Testing methodology	Focus on acceptance criteria from start	No mature QA	SaaS/enterprise
35	Model Driven Development (MDD)	Model-based	Generating code from models, standard repetitive systems	Custom UI-heavy products	Telecom, industrial systems
36	Secure SDLC (SSDLC)	Security process	Compliance, secure-by-design software	No security budget	Banking, government
37	CMMI-based Process	Maturity framework	Process maturity, repeatability important	Startup, fast agile teams	IT service companies
38	ITIL-driven Delivery	Service management	IT services and operations governance	Startup product build	IT operations/infra services
39	PRINCE2	Project management methodology	Strong governance & stage control required	Small fast teams	Government projects
40	Hybrid (Agile + Waterfall)	Mixed methodology	Some parts fixed + some flexible	High confusion / unclear roles	Enterprise, regulated + agile teams

❑ **Quick Selection Guide (Super Practical)**

✔ If your project type is...

Project situation	Best methodology
Requirements are fixed & compliance heavy	Waterfall / V-Model
Risk is high and complex	Spiral
Requirements unclear & UI needs validation	Prototype Model / RAD
Fast delivery with changing requirements	Agile / Scrum
Continuous work (support + bug fixing)	Kanban / Scrumban
Large enterprise (many teams)	SAFe / LeSS / Nexus
Frequent deployments needed	DevOps / CI/CD
Security must be integrated	DevSecOps / SSDLC
Complex business domain	DDD + Agile (Scrum)
High code quality needed	XP + TDD
IT operations/service environment	ITIL
Strong governance project mgmt	PRINCE2 / CMMI

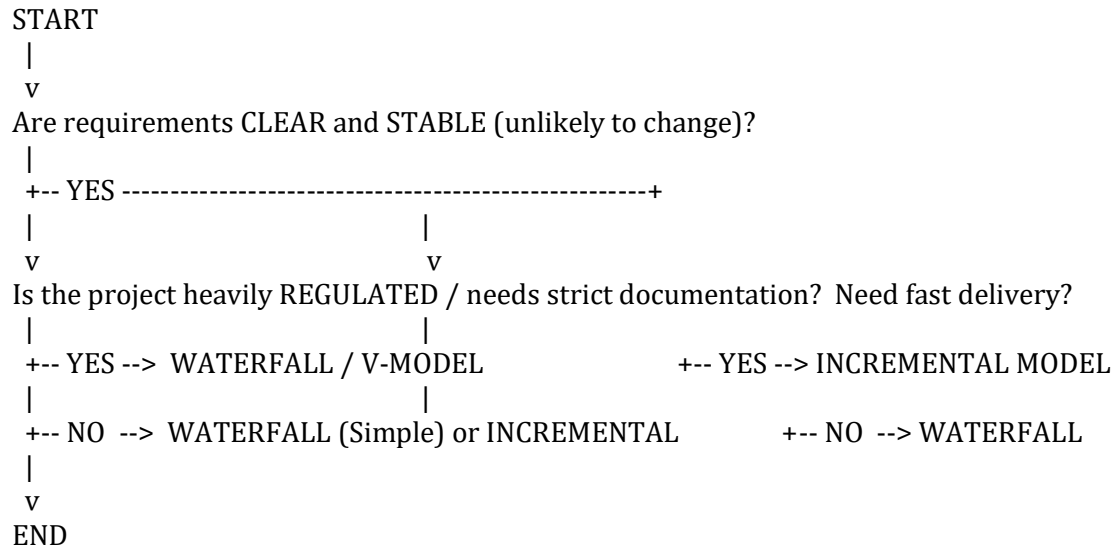
If you want, I can also create:

- ✔ **One-page printable PDF cheat sheet** (Methodology → When to Use)
- ✔ **Decision flowchart** ("If requirements are stable → choose X else choose Y")
- ✔ **Methodology comparison table** (Agile vs Waterfall vs Spiral vs DevOps)

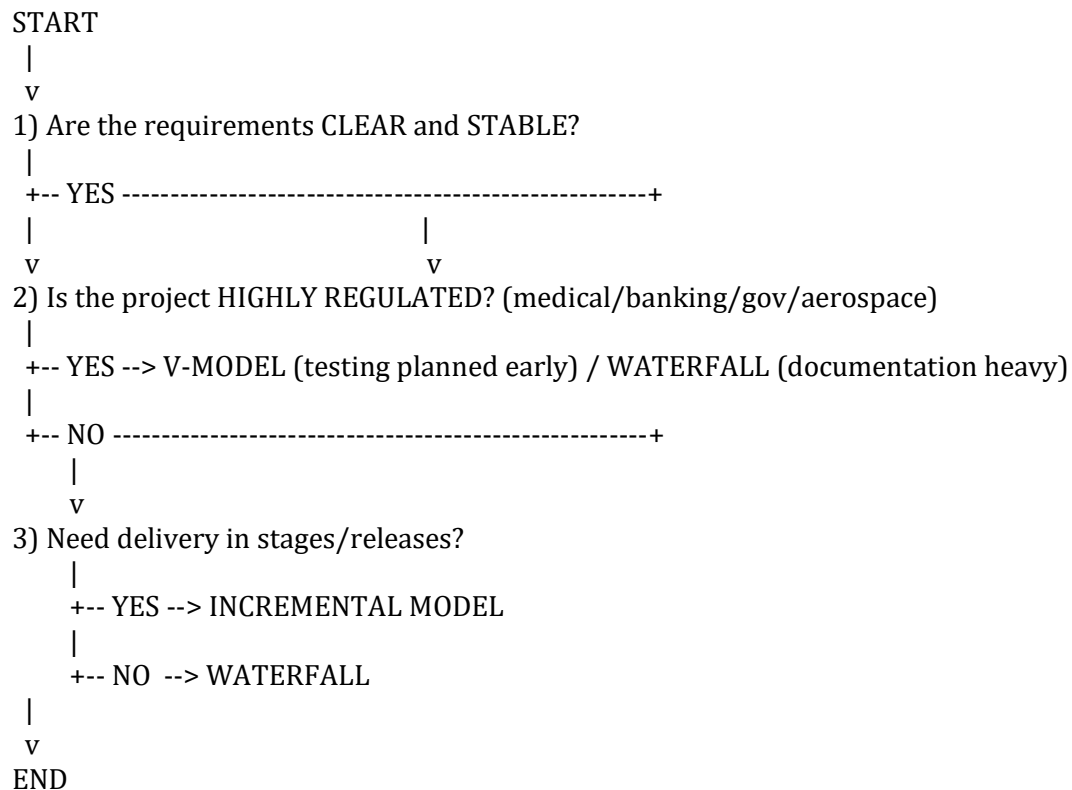
Absolutely ✔

Here is a **Decision Flowchart** to choose the **right software methodology** based on your project situation:

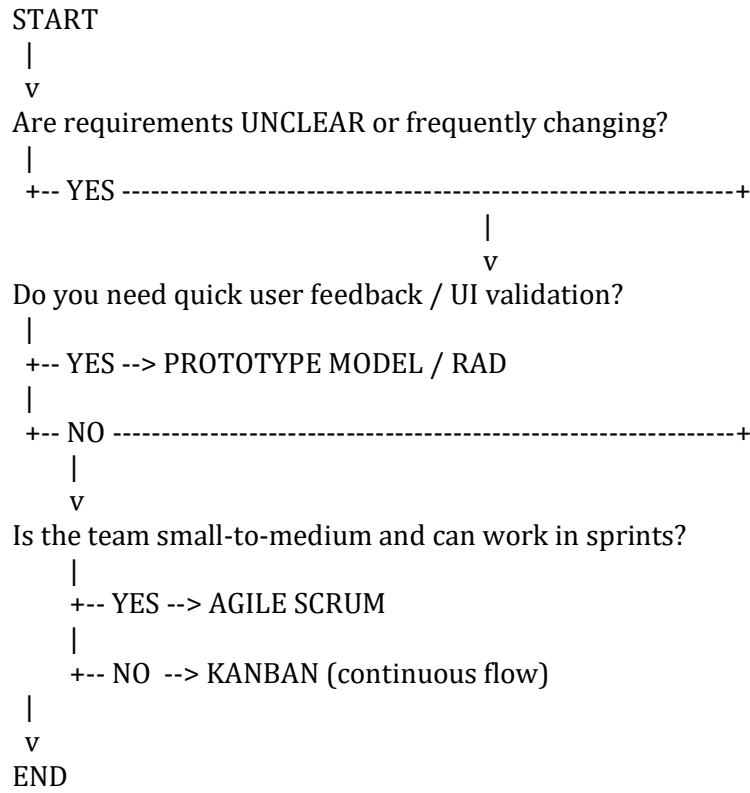
✔ Methodology Decision Flowchart (Best Fit Selection)



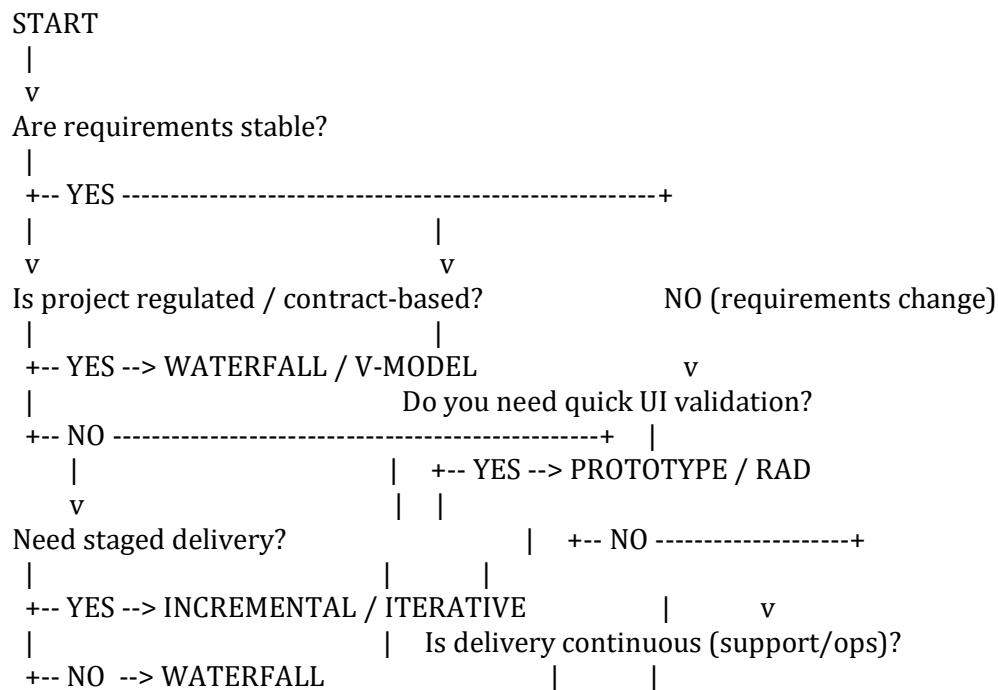
❑ Full Practical Flowchart (Enterprise Grade)

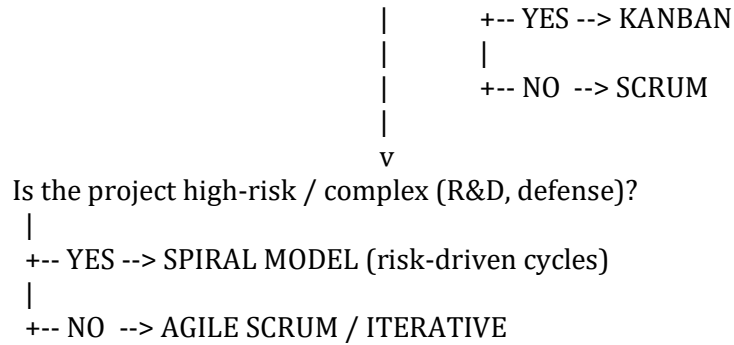


✔ If Requirements are NOT Stable / Unclear (Most real-world projects)



❑ Complete Industry Methodology Decision Flowchart (Best One)





⚡ Special Case Routing (Must know)

Need frequent deployments + automation?
--> DevOps / CI-CD

Need security in every pipeline stage?
--> DevSecOps / SSDLC

Many teams (large enterprise scaling)?
--> SAlFe / Nexus / LeSS

Complex business logic (banking, insurance)?
--> DDD + Agile Scrum

Need best code quality?
--> XP + TDD

If you want, I can also convert this into a **professional diagram image (flowchart)** that you can directly paste into PPT/Word/Project report.

Sure ✔ Here is the **MASTER comparison table** for **Agile vs Waterfall vs Spiral vs DevOps** (industry style).

✔ Methodology Comparison Table (Agile vs Waterfall vs Spiral vs DevOps)

Factor	Waterfall	Agile (Scrum/Kanban)	Spiral	DevOps
Category	Traditional / Sequential SDLC	Iterative & Incremental SDLC	Risk-driven SDLC	Delivery + Operations methodology

Main Goal	Deliver complete project in fixed phases	Deliver value continuously in iterations	Reduce risk through repeated cycles	Fast reliable release + automation + collaboration
Approach	Step-by-step phases (Req → Design → Dev → Test → Deploy)	Iterations (Sprint-based / flow-based)	Cycles: Planning → Risk Analysis → Build → Evaluate	CI/CD, automation, monitoring, Infra as Code
Requirement Handling	Must be clear & stable from start	Changes are expected and welcomed	Requirements evolve per cycle	Continuously evolving with feedback
Customer Involvement	Low (mostly at start/end)	Very High (continuous feedback)	High (review each cycle)	High (feedback via production monitoring)
Delivery Style	One final delivery	Frequent delivery (every sprint/week)	Multiple prototypes/releases per cycle	Continuous delivery / deployment
Change Management	Expensive and difficult	Easy and flexible	Flexible but structured	Very flexible with automation
Risk Handling	Weak (risk discovered late)	Medium (risks discovered early in sprints)	Strongest (risk is primary focus)	Strong (release risks reduced by automation/testing)
Testing	Mostly at the end	Continuous testing in each iteration	Testing in every cycle	Automated testing in pipelines (CI)
Documentation Level	Very High	Medium / Light	High	Medium (docs + automated runbooks)
Speed of Delivery	Slow	Fast	Medium	Very Fast
Cost Predictability	High (if scope stable)	Medium (scope flexible)	Medium (depends on risks)	Medium (depends on automation maturity)

Quality	Medium (bugs found late)	High (continuous testing & review)	Very High (risk + validation each cycle)	Very High (automation + monitoring)
Best for	Fixed-scope projects	Product development, changing requirements	Complex, high-risk, costly projects	Cloud apps, microservices, frequent releases
Worst for	Rapidly changing requirements	Strict contracts with fixed scope	Small/low budget/simple projects	Teams without automation culture
Team Size Fit	Medium-large	Small-medium (can scale)	Medium-large	Any, but best for medium-large
Project Control	High control via phases	High control via backlog + sprints	High control via risk management	High control via pipelines & monitoring
Typical Tools	MS Project, Visio, Docs	Jira, Trello, Azure Boards	Risk logs, prototypes	Git, Jenkins/GitHub Actions, Docker, Kubernetes
Examples	Govt tenders, fixed ERP	Startups, SaaS apps	Defense, aerospace, big R&D	Netflix-style systems, fintech CI/CD

❑ Quick Decision Guide (1-Line Summary)

If your project needs...	Choose...
Fixed scope, strict documentation	✓ Waterfall
Fast delivery + changing requirements	✓ Agile
High risk & high complexity	✓ Spiral
Continuous delivery + automation	✓ DevOps

✓ Best Combination Used in Real Companies

In modern IT, the most successful approach is often:

✓ Agile + DevOps

- Agile = *how you build software*
- DevOps = *how you release and operate software*

If you want, I can also create:

✓ **Agile vs DevOps vs DevSecOps comparison table** (very commonly asked in interviews).

Here is the **most asked interview comparison table**:

✓ **Agile vs DevOps vs DevSecOps — Comparison Table (Interview Master Table)**

Factor	Agile	DevOps	DevSecOps
Meaning	A methodology for software development using iterations	A culture/practice to combine Dev + Ops for faster delivery	DevOps + Security integrated into every stage
Primary Focus	Faster feature development + adaptability	Faster & reliable releases + automation	Secure + compliant releases with automation
Main Goal	Deliver working software frequently with feedback	Continuous delivery with stability	Continuous delivery with built-in security
Works Between	Business ↔ Developers ↔ QA	Developers ↔ Operations ↔ QA	Developers ↔ Ops ↔ Security ↔ Compliance
Core Idea	“Build in small increments”	“Automate & deploy continuously”	“Security is everyone’s responsibility”
Lifecycle Stage Covered	Mostly planning → development → testing	Mostly build → release → deploy → monitor	Entire SDLC + security checks in each stage
Security Approach	Usually security considered later (optional)	Security sometimes added but not always	Security is mandatory & integrated from start
Testing	Continuous testing per sprint	Automated tests in pipelines (CI)	Automated tests + security tests (SAST/DAST/SCA)

Deployment	Frequent releases (weekly/bi-weekly)	Continuous deployment (daily/hourly possible)	Continuous deployment but only after security gates
Speed	Fast development cycles	Fast delivery cycles	Fast + controlled with security
Risk Handling	Risk reduced by frequent feedback	Risk reduced by automation & monitoring	Risk reduced by automation + security scanning
Compliance Fit (Banking/Gov)	Medium	Medium	High (Best fit)
Best For	Changing requirements, product development	Cloud apps, microservices, CI/CD	Banking/fintech, regulated industries, security-critical apps
Challenges	Scope creep, poor backlog handling	Needs automation maturity + culture change	Needs security tools + security mindset
Key Metrics	Velocity, sprint burndown, lead time	Deployment frequency, MTTR, failure rate	Vulnerability count, patch time, compliance pass rate
Most Common Tools	Jira, Azure Boards, Trello	Git, Jenkins/GitHub Actions, Docker, Kubernetes	Same as DevOps + Snyk, SonarQube, OWASP ZAP, Trivy, Vault
Common Interview Line	"Agile improves development process"	"DevOps improves deployment and operations"	"DevSecOps shifts security left in CI/CD"

✔ Super Short Interview Answers (1-2 lines)

✔ Agile

Agile is a development methodology for delivering features faster using iterative sprints and continuous customer feedback.

✔ DevOps

DevOps is a culture + set of practices that unifies development and operations to enable fast, reliable and automated software delivery.

✔ DevSecOps

DevSecOps is DevOps with security integrated into every step of SDLC through automated security testing and compliance controls.

☐ **Most Important Interview Difference (Golden Line)**

- ✓ **Agile = How you develop software**
 - ✓ **DevOps = How you deliver & run software**
 - ✓ **DevSecOps = How you deliver & run software securely**
-

If you want, I can also give you:

- ✓ **Agile vs Scrum vs Kanban table** (another top interview topic)